SETUP GUIDE

Your Own Discord Music Bot

How to create the bot, add it to your server, put it online on Render (free) and keep it running for you and your friends.

What you need	
A Discord account	with **Manage Server** permission on your server (you own it, or you're an admin)
A GitHub account	Render deploys the bot straight from GitHub
A Render account	free at render.com (sign up with GitHub)
About 20 minutes	no coding needed

Code: https://github.com/muhammad-propfirms/discord-music-bot

Version 5.1, November 2026

OVERVIEW

The big picture

You will do five things. Each one is a section in this guide:

Part	What you do	Time
1	Create the bot on the Discord Developer Portal and copy its **token** (the bot's password)	5 min
2	Invite the bot to your server	2 min
3	Copy your **Server ID** (optional, but it makes commands appear right away)	1 min
4	Deploy on **Render** using the Blueprint in the repo, and fill in the environment variables	8 min
5	Keep it awake 24/7 with a free uptime monitor	3 min

After that, see **Part 6** (what to do if YouTube blocks Render), **Part 7** (all commands for your friends), **Part 8** (running it on your own PC instead) and **Part 9** (troubleshooting).

> **How the bot works (in one paragraph)**
> Someone types `/play never gonna give you up`. The bot looks the song up with **yt-dlp**, has yt-dlp download the audio, converts it with **ffmpeg** and streams it into your voice channel. yt-dlp and ffmpeg are installed automatically, and yt-dlp updates itself every time the bot starts. That matters because YouTube changes often, and an outdated yt-dlp is the most common reason music bots stop working.

What was broken in the original bot

- **No audio at all:** Discord now requires end-to-end encrypted voice ("DAVE") and newer encryption. The old voice library could not connect. It is upgraded now.
- **YouTube returned "403 Forbidden"** when ffmpeg tried to read the audio directly. The audio now goes through yt-dlp itself, using a small private relay inside the bot.
- **yt-dlp needs a JavaScript runtime** for YouTube now. The bot passes Node.js to it automatically.
- **ffmpeg was never configured**, a privileged intent made logins fail, and some commands crashed by replying twice.
- **Real YouTube login cookies** were committed in the old repo. They have been removed from the code and from the history of the new repo.

PART 1

Create the bot on Discord

Every Discord bot is an "application" with a secret token. The token is what lets the program log in as the bot.

1. Open discord.com/developers/applications and log in with your normal Discord account.
2. Click **New Application** (top right). Give it a name, such as *Party DJ*, accept the terms and click **Create**.
3. Optional: on **General Information**, upload an avatar picture and click **Save Changes**.
4. In the left menu, open **Bot**. Change the username if you like.
5. Under **Token**, click **Reset Token** (confirm, and enter your 2FA code if asked), then click **Copy**. Paste it somewhere private for now, such as a notes app. **You only see it once.**
6. Still on the **Bot** page, scroll to **Privileged Gateway Intents**. This bot needs **none** of them, so you can leave all three switched off.
7. Recommended: make the bot private, so only you can add it to servers. Open **Installation**, set **Install Link** to **None** and save. Then go back to **Bot**, switch **Public Bot** off and save. (Discord won't let you turn off Public Bot until the install link is set to None.)

Treat the token like a password
Anyone with the token can control your bot. Never post it in Discord, in a screenshot, or in GitHub. If it leaks, go to **Bot > Reset Token**. The old token stops working immediately. Then put the new token into Render (Part 4).

What the token looks like
A long string of letters and numbers with two dots in it, roughly 70 characters, for example `MTIzNDU2Nzg5MDEyMzQ1Njc4.GaBcDe.xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx`. Copy only that, with no spaces before or after it, no quotes, and no word "Bot" in front of it.

PART 2

Invite the bot to your server

You create an invite link that asks for exactly the permissions a music bot needs, then open it in your browser.

1. In the Developer Portal, open **OAuth2** and scroll down to **OAuth2 URL Generator**.
2. Under **Scopes**, tick **bot** and **applications.commands**.
3. A **Bot Permissions** box appears below. Tick: **View Channels**, **Send Messages**, **Embed Links**, **Read Message History**, **Connect** and **Speak**.
4. Leave **Integration Type** on **Guild Install**. Copy the **Generated URL** at the bottom.
5. Open that URL in your browser, choose your server in the dropdown, click **Continue** and then **Authorize**.
6. The bot now shows in your server's member list as **offline**. That is expected. It comes online after Part 4.

Shortcut
You can also build the link by hand. Copy your **Application ID** from **General Information** and put it into:
`https://discord.com/oauth2/authorize?client_id=YOUR_APP_ID&permissions=3230720&scope=bot%20applications.commands`
The bot also prints this exact link in its logs at startup if it isn't in any server yet.

Don't see your server in the dropdown?
You need the **Manage Server** permission on that server. Ask the owner to invite the bot using your link.

PART 3

Copy your Server ID (recommended)

Slash commands registered for a single server appear right away. Without a Server ID, the bot registers its commands in every server it's in, which also works.

1. In the Discord app, open **User Settings** (the gear next to your name) and go to **Advanced**.
2. Switch on **Developer Mode**.
3. Close settings, **right-click your server's icon** in the left sidebar and click **Copy Server ID**. It is a long number, such as `1098765432109876543`.

Optional: to let only certain people use the bot, open **Server Settings > Roles**, right-click a role (for example *DJ*) and choose **Copy Role ID**. You'll need it for `ALLOWED_ROLE_IDS`.

PART 4

Put the bot online with Render

The repo contains a file called render.yaml (a "Blueprint"). When you give Render the repo, it reads that file and sets everything up for you: the service type, the build and start commands, the health check, and a form for the environment variables.

4.1 Create the service from the Blueprint

1. Go to render.com and sign up or log in with **GitHub**.
2. If Render asks, allow it to access your GitHub repositories. Choose **All repositories**, or select **discord-music-bot**.
3. In the Render dashboard, click **+ New** (top right), then **Blueprint**.
4. Find **discord-music-bot** in the list and click **Connect**. If it's missing, use **Configure account** to give Render access to it.
5. Give the Blueprint a name, such as *music-bot*. Render shows what it will create: one **Web Service** called **discord-music-bot** on the **Free** plan. You don't need to choose a service type. The Blueprint already did that.
6. Fill in the environment variables shown on the page (next section), then click **Deploy Blueprint** (on some accounts the button says **Apply**).
7. Open the new **discord-music-bot** service and click **Logs**. The first build takes 2 to 4 minutes.

> **Even faster**
> The README on GitHub has a **Deploy to Render** button. It opens the same Blueprint screen directly.

4.2 Fill in the environment variables correctly

Render asks only for the values it can't know. Type them **exactly** as described:

Key	Needed?	What to paste	Example
TOKEN	**Yes**	The bot token from Part 1, step 5. The token only: no quotes, no spaces, no "Bot " in front.	`MTIz...xxxx`
GUILD_ID	Optional	Your Server ID from Part 3. For several servers, separate them with commas and no spaces. Leave it empty to use every server the bot is in.	`1098765432109876543`
ALLOWED_ROLE_IDS	Optional	Role ID(s) allowed to use the bot, comma-separated. **Leave empty** so everyone can use it.	*(empty)*

These are filled in for you, and you can change them later:

Key	Default	Meaning
NODE_VERSION	24	Node.js version Render uses. Leave it alone.
DEFAULT_VOLUME	50	Starting volume for each new queue (0 to 100)
MAX_PLAYLIST_SIZE	100	The maximum number of songs taken from one playlist link
LEAVE_WHEN_ALONE_AFTER	60	Seconds before the bot leaves once everyone has left the voice channel (0 = never)
LEAVE_WHEN_IDLE_AFTER	300	Seconds before the bot leaves after the queue finishes (0 = never)
YTDLP_AUTO_UPDATE	true	Update yt-dlp on every start. Keep this **true**.
PORT	*set by Render*	**Never add this.** Render sets it to 10000 by itself. A wrong PORT value makes the deploy hang with *No open ports detected*. (Since v5.1 the bot ignores a bad value on Render, but delete it anyway.)
SOUNDCLOUD_FALLBACK	true	If YouTube blocks Render, play the same song from SoundCloud instead of failing

The 5 most common env-var mistakes
1. Quotes around the value (`"MTIz..."`). Render stores the quotes too, and the login fails.
2. A space or line break copied along with the token. Paste into a plain-text note first to check.
3. Putting the **Application ID** or **Client Secret** in TOKEN. It must be the **Bot** token.
4. A server *name* or invite link in GUILD_ID. It must be the long number.
5. Adding a **PORT** variable yourself. Render sets it automatically, and a wrong value breaks the health check.

4.3 Changing a variable later

1. Render dashboard > **discord-music-bot** > **Environment**.
2. Click **Edit**, change the value (or **+ Add Environment Variable**) and click **Save, rebuild, and deploy** (or **Save and deploy**).
3. Wait about 2 minutes for the restart. Anything in the queue at that moment is cleared.

4.4 Check that it worked

In **Logs**, you should see lines like these. The bot is then online in Discord with the status *Listening to /play*:

```
[WEB] Health check on port 10000
[yt-dlp] Ready (version 2026.10.26)
[COOKIES] Loaded 23 YouTube/Google cookies from /etc/secrets/cookies.txt (logged-in account).
[STREAM] Local audio relay listening on 127.0.0.1:41235
[BOT] Logged in as Party DJ#1234 (id 1234567890)
[BOT] In 1 server(s): Our Friends Server
[COMMANDS] Registered 20 slash commands in "Our Friends Server".
==> Your service is live
[YOUTUBE] Test passed: this server can play YouTube.
```

Now join a voice channel in Discord and type `/play` followed by a song name. If the commands don't show up, press **Ctrl+R** to reload Discord.

PART 5

Keep it awake 24/7 (free plan)

Render's free web services go to sleep after 15 minutes with no web traffic. A sleeping bot shows as offline in Discord. Talking to the bot on Discord does not wake it, but a web request does.

The fix is a free uptime monitor that visits the bot's health page every 5 minutes, so it never falls asleep:

1. In Render, open your service and copy its URL from the top of the page, for example `https://discord-music-bot-abcd.onrender.com`.
2. Add `/healthz` to the end of it. Opening it in your browser should show `{"status":"ok","discord":"connected",...}`.
3. Create a free account at uptimerobot.com (cron-job.org or Better Stack also work).
4. Click **New monitor**, choose type **HTTP(s)**, paste the `/healthz` URL and set the interval to **5 minutes**. Save.

Is this allowed and free?
Yes. Render's free plan includes 750 instance hours per month, which covers one service running all month (about 730 hours). Don't run a second free service on the same account, or you'll run out before the month ends.

Want zero hassle?
Upgrade the service to Render's **Starter** plan (about $7/month): **Settings > Instance Type**. It never sleeps and has more CPU and RAM, so playback is smoother. You can then delete the uptime monitor.

Updates and restarts

- Render redeploys automatically whenever new code is pushed to the GitHub repo (`autoDeploy: true`).
- To restart by hand: **Manual Deploy > Deploy latest commit**, or **Clear build cache & deploy** if something seems stuck.
- Every restart downloads the newest yt-dlp. If YouTube suddenly stops working, try a restart first.

PART 6

If YouTube blocks Render ("confirm you're not a bot")

YouTube often blocks big cloud providers' IP addresses, including Render's. The logs then say [YOUTUBE] Test FAILED: YouTube is blocking this server. Music keeps working, because the bot automatically plays the same song from SoundCloud and posts a short notice in chat. To get YouTube itself working, give yt-dlp the cookies of a logged-in YouTube account.

Use a spare Google account
Cookies work like a logged-in session. Use a **throwaway or secondary** Google account, not your main one. Never commit the cookies file to GitHub or share it in Discord.

1. In Chrome or Firefox, open a **private/incognito window** and log in to youtube.com with the spare account.

2. In the same tab, go to `https://www.youtube.com/robots.txt`.
3. Export the cookies in **Netscape cookies.txt format** with an extension such as **Get cookies.txt LOCALLY** (Chrome) or **cookies.txt** (Firefox). Save the file as `cookies.txt`.
4. **Close the private window right away** without logging out. Logging out would invalidate the cookies.
5. In Render, open the service > **Environment** > **Secret Files** > **+ Add Secret File**.
6. Set **Filename** to exactly `cookies.txt`, paste the whole contents of the file and click **Save**, then redeploy.
7. After the deploy, the logs say `[COOKIES] Loaded ... cookies ... (logged-in account)` and, a few seconds after login, `[YOUTUBE] Test passed`. The bot picks up `/etc/secrets/cookies.txt` by itself, so you don't need any extra variable.

Log line	Meaning / what to do
`[COOKIES] Loaded N ... (logged-in account)`	Good. Cookies were read correctly.
`[COOKIES] Tabs had been turned into spaces`	Fine. The bot repaired the copy/paste damage itself.
`[COOKIES] ... JSON format`	Wrong export type. Export again as **Netscape / cookies.txt**.
`[COOKIES] ... do not include a login`	You exported while logged out. Log in to YouTube first, then export again.
`[COOKIES] No youtube.com cookies`	You exported cookies of a different site. Export while the YouTube tab is open.
`[YOUTUBE] Test passed`	YouTube works on this server.
`[YOUTUBE] Test FAILED ... blocking`	Cookies missing, expired or rejected. Export fresh ones. Meanwhile songs play from SoundCloud.

Cookies eventually expire. If the error comes back after a few weeks, repeat these steps. If YouTube keeps blocking you, running the bot from a home PC (Part 8) avoids the problem, because home internet IPs are rarely blocked.

PART 7

Using the bot (share this with your friends)

Join a voice channel first, then type / in any text channel to see the commands.

Command	What it does	Example
/play query	Plays a song by name or link, or adds it to the queue. Works with YouTube, YouTube Music, SoundCloud, Bandcamp, playlist links and Spotify *track* links.	`/play blinding lights`
/play query next:True	Adds the song to the **front** of the queue	`/play daft punk next:True`
/search query	Shows 5 results and lets you click one	`/search lofi`
/radio query [songs]	Type a song, artist or mood. Plays the best match plus related songs (YouTube's Mix for it), then keeps going with autoplay	`/radio 90s rock`
/autoplay [mode]	When the queue runs out, keep playing songs related to the last one. On, off, or toggle	`/autoplay mode:On`
/pause /resume	Pauses or continues playback	
/skip	Plays the next song	
/previous	Plays the previous song again	
/queue [page]	Shows the queue with numbers	`/queue page:2`
/jump position	Skips ahead to song number N in /queue	`/jump position:5`
/remove position	Removes song number N from the queue	`/remove position:3`
/nowplaying	Shows the current song with a progress bar	
/loop mode	Off, This song, or Whole queue	`/loop mode:Whole queue`
/shuffle	Shuffles the upcoming songs	
/seek time	Jumps to a time in the current song	`/seek time:1:30`
/volume [percent]	Shows or sets the volume (0 to 150)	`/volume percent:70`
/stop	Stops, clears the queue and leaves	
/leave	Makes the bot leave the voice channel	
/help /info	Lists commands; shows status, ping and versions	

- **Just type what you want.** `/play` accepts a song title or any text, such as `/play shape of you`, and plays the best YouTube match. `/radio` does the same and also lines up related music that fits what you typed.
- Only people in the **same voice channel** as the bot can control it while it's playing.
- A new song takes about 5 to 10 seconds to start. Later songs in the queue start almost instantly, because the bot preloads the next one.

- Links work from YouTube, YouTube Music, SoundCloud, Bandcamp, Spotify (tracks), Vimeo, Twitch, Mixcloud, Audiomack, Deezer and Dailymotion. Other websites are blocked for security. The owner can allow more with the `EXTRA_ALLOWED_HOSTS` variable (comma-separated domains).
- Spotify *playlists* can't be played (Spotify audio is DRM-protected). Use a YouTube or SoundCloud playlist link instead.

PART 8

Alternative: run it on your own PC

Useful for testing, or if YouTube keeps blocking Render. The bot only runs while the PC is on.

1. Install **Node.js 24 LTS** from nodejs.org (keep all the default options) and **Git** from git-scm.com.
2. Open **Terminal** (Windows: right-click Start > Terminal) and run:

```
git clone https://github.com/muhammad-propfirms/discord-music-bot.git
cd discord-music-bot
npm install
copy .env.example .env        (Mac/Linux: cp .env.example .env)
notepad .env                  (Mac: open -e .env)
```

3. In the `.env` file, set `TOKEN=` to your token (no quotes), set `GUILD_ID=` if you like, and save.
4. Run the self-test. It checks Node, ffmpeg, yt-dlp, real YouTube playback and your token, and prints your invite link:

```
npm run check
```

5. When it says **All checks passed**, start the bot. Keep the window open while you're using it:

```
npm start
```

Optional: to keep it running in the background and restart it automatically, use pm2:

```
npm install -g pm2
pm2 start index.js --name music-bot
pm2 logs music-bot        (view logs)
pm2 save                   (remember it across restarts)
```

PART 9

Troubleshooting

Check Render's Logs tab first. Most problems are explained there.

Symptom	Likely cause	Fix
Bot stays offline; log says *Discord rejected the token*	Wrong or old token, quotes, or spaces	Reset the token (Part 1), paste it again into TOKEN without quotes, then save and redeploy
Bot was online, now offline (free plan)	Render put it to sleep	Set up the uptime monitor (Part 5), or upgrade to Starter
No slash commands in Discord	Bot invited without **applications.commands**, wrong GUILD_ID, or Discord hasn't refreshed	Press Ctrl+R in Discord. Check that GUILD_ID is your server's ID. Re-invite with both scopes (Part 2)
Log: *Missing access to server ...*	GUILD_ID is a server the bot isn't in	Invite the bot to that server, or clear GUILD_ID
Deploy stuck on *In progress*; logs repeat *No open ports detected on 0.0.0.0*	A **PORT** variable with a wrong value	Environment > delete **PORT** > save. Then **Manual Deploy > Clear build cache & deploy**
"YouTube is blocking this server" / songs play from SoundCloud	Render's IP is flagged by YouTube	Add fresh cookies (Part 6), or run it at home (Part 8)
"Could not play ...: ffmpeg exited with code 183"	The song's audio download failed (usually the YouTube block above)	Read the message just before it, and fix that. Update to the latest code: it plays from SoundCloud instead
Changes in Environment don't seem to apply	The new deploy never finished, so the old instance is still running	Open **Deploys**: the newest one must say **Live**. Cancel stuck ones and deploy again
Bot joins but there's no sound	Missing Speak permission, or a stalled stream	Give the bot **Connect** and **Speak** on that channel. Try /skip, then a restart. Run `npm run check` locally to test
"I don't have permission to join or speak"	Channel permissions override the bot's role	Edit the channel > Permissions > add the bot > allow Connect and Speak
A song is "unavailable" / "private"	The video is blocked, deleted or region-locked	Pick another upload of the song
Age-restricted video won't play	YouTube requires a login	Add cookies from an adult account (Part 6)
Sound stutters	The free instance has very little CPU	Upgrade to Starter, or run at home
Deploy fails at *npm ci*	package-lock.json missing or out of date	Use the repo as-is. If you changed dependencies, run npm install and push the new lock file
Everything worked, then YouTube broke for everyone	YouTube changed something	Restart the service (it updates yt-dlp). Fixes usually ship within a day or two

Still stuck? Set the environment variable `DEBUG_DISTUBE` to `true`, redeploy, and the logs will show every step of playback.

Security checklist

- Tokens and cookies go only in Render's Environment and Secret Files, or in your local `.env`. The repo's `.gitignore` keeps `.env` and `cookies.txt` out of git.
- Keep **Public Bot** off so strangers can't add your bot to their servers.
- If you think the token leaked: **Reset Token** right away and update TOKEN in Render.